

16주차

이미지 분류(Image Classification)란?

이미지 분류란 이미지 안에 있는 객체(Object) 또는 장면(Scene) 을 인식하여, 사전에 정의된 클래스(label) 중 하나 또는 여러 개로 이미지를 구분하는 문제이다.

- 지도 학습(Supervised Learning)의 대표적인 문제
- 입력: 이미지
- 출력: 클래스(label)

컴퓨터 비전 분야에서 가장 기본이자 핵심적인 문제로, 이후의 객체 탐지(Object Detection), 세그멘테이션(Segmentation)의 기초가 된다.

이미지 분류의 종류

단일 클래스 분류 (Single Classification)

- 이미지 전체를 하나의 대표 클래스로만 분류
- 이진 분류(Binary Classification) 형태

예시

- "이 이미지는 개인가?" → 예 / 아니오
- 출력:
 - 개 (90%)

이미지 안에 여러 객체가 있어도 하나만 선택

다중 클래스 분류 (Multi-Class Classification)

- 여러 클래스 중 하나만 선택
- 클래스들은 서로 배타적

예시

- 개 / 고양이 / 늑대 중 하나
- 출력 예:
 - 로트와일러 80%
 - 도베르만 12%
 - 리트리버 8%

가장 확률이 높은 **하나의 클래스**가 최종 결과

다중 레이블 분류 (Multi-Label Classification)

- 하나의 이미지에서 여러 클래스를 동시에 예측
- 클래스들이 서로 독립적

예시

- 이미지 안에:
 - 개 (98%)
 - 소파 (82%)
 - 블라인드 (51%)

하나의 이미지에 **여러 의미 있는 라벨**을 동시에 부여

다중 레이블 분류는

- 더 많은 데이터
 - 더 복잡한 모델 구조
- 를 요구함
-

이미지 분류에 사용되는 알고리즘

전통적인 방법부터 딥러닝까지 다양함:

1. SVM (Support Vector Machine)
2. KNN (K-Nearest Neighbors)
3. Decision Tree
4. ANN (Artificial Neural Network)
5. **CNN (Convolutional Neural Network)**

AlexNet 개요

AlexNet은

- 2012년 ILSVRC(ImageNet) 대회 우승 모델
- CNN이 기존 방법을 압도했음을 증명한 모델

성과

- 기존 얇은 구조: 오류율 약 26%
- AlexNet: 오류율 약 16%

딥러닝 기반 CNN 시대의 시작

AlexNet 구조의 핵심 아이디어

기본 흐름

입력 이미지

- 합성곱(Convolution)
- 활성화 함수(ReLU)
- 풀링(Max Pooling)
- 완전 연결 계층(Fully Connected)
- 클래스 출력

특징

- 앞쪽 계층: 공간 크기 감소
 - 뒤쪽 계층: 채널(특징 맵) 수 증가
 - 표현력은 높이고 연산량은 줄이는 구조
-

LeNet-5 vs AlexNet 비교

LeNet-5 (1998)

- 입력: 28×28 흑백 이미지
- 구조:
 - 합성곱 2개
 - 평균 풀링
 - Sigmoid / Tanh 활성화
- 얇은 구조

AlexNet (2012)

- 입력: 224×224 RGB 이미지
- 구조:
 - 합성곱 5개
 - 최대값 풀링(Max Pooling)
 - **ReLU 활성화**
 - **Dropout 사용**
- 훨씬 깊고 큰 모델

핵심 차이점

구분	LeNet-5	AlexNet
활성화 함수	Sigmoid	ReLU
풀링 방식	평균 풀링	최대값 풀링
입력 크기	작음	큼
깊이	얇음	깊음
과적합 대응	없음	Dropout

ReLU의 장점

- 기울기 소실 문제 완화
- 깊은 네트워크 학습 가능

Dropout의 역할

- 학습 중 일부 뉴런을 **확률적으로 비활성화**
- 모델이 특정 뉴런에 의존하지 않게 함
- **과적합(overfitting) 방지**

특히 매개변수가 많은 **완전 연결 계층**에서 효과적

사전 학습된 AlexNet 모델

PyTorch에서 AlexNet 불러오기

```
model = torchvision.models.alexnet(  
    weights="AlexNet_Weights.IMAGENET1K_V1"  
)
```

- ImageNet(1000 클래스)으로 사전 학습됨
- weights=None → 랜덤 초기화

AlexNet 성능 지표

acc@1, acc@5

- **acc@1**: 가장 높은 확률의 클래스가 정답일 확률
- **acc@5**: 상위 5개 예측 중 정답 포함 여부

예:

```
고양이 50%  
말 30%  
소 10%  
개 5%  
늑대 1%
```

- acc@1
- acc@5

GFLOPS

- 초당 수행 가능한 부동소수점 연산 수
- 값이 클수록 연산량이 많고 모델이 큼

torchinfo.summary로 모델 구조 확인

```
summary(model, (1, 3, 224, 224))
```

확인 가능 정보:

- 각 계층의 출력 크기(Output Shape)
- 계층별 매개변수 수(Param #)
- 전체 파라미터 수 (약 6,100만 개)

완전 연결 계층이 **파라미터 대부분**을 차지

AlexNet에 사용된 주요 계층 정리

계층	설명
Conv2d	합성곱 연산
ReLU	비선형 활성화
MaxPool2d	최대값 풀링
AdaptiveAvgPool2d	출력 크기 고정
Dropout	과적합 방지
Linear	완전 연결 계층
Sequential	계층 묶음

ImageNet 클래스 정보

- ImageNet: **1000개** 클래스
- 클래스 이름은 `imagenet_classes.txt` 로 제공

```
with open("imagenet_classes.txt") as f:
    classes = f.read().splitlines()
```

AlexNet 입력 이미지 전처리

AlexNet은 학습 당시 사용한 전처리 방식과 동일하게 입력해야 함

```
transforms.Compose([
    Resize((224, 224)),
    ToTensor(),
    Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])
```

이유

- 학습 데이터 분포와 입력 분포를 맞추기 위해

모델 추론(Inference)이란?

모델 추론이란

이미 학습이 끝난 모델에 **새로운 입력 이미지**를 넣어

클래스 예측 결과와 확률을 계산하는 과정이다.

- 학습(training)이 아님
- 가중치(weight)는 **변경되지 않음**
- 순전파(forward pass)만 수행

```
with torch.no_grad():
```

- 기울기 계산 비활성화
- 메모리 절약 + 연산 속도 향상
- 추론 시 필수

AlexNet 추론 코드 흐름 정리

(1) 모델 출력

```
outputs = model(tensors.to(device))
```

- 출력 형태: `(batch_size, 1000)`
- ImageNet 1000개 클래스에 대한 **logit 값**

(2) Softmax → 확률 변환

```
probs = F.softmax(outputs, dim=-1)
```

- 각 클래스의 확률 분포 생성
- 모든 확률의 합 = 1

(3) Top-k 예측

```
top_probs, top_idxes = probs.topk(5)
```

- `top_probs`: 가장 높은 확률 5개
- `top_idxes`: 해당 클래스 인덱스

(4) 사람이 읽기 쉬운 형태로 변환

```
top_classes = np.array(classes)[top_idxes]
```

- 클래스 인덱스 → 클래스 이름 매핑

(5) 결과 해석 예시

- airplane 이미지
 - airliner 73%
 - warplane 14%

- bus 이미지
 - streetcar 60%
 - trolleybus 37%

모델이 시각적으로 유사한 클래스들을 함께 높은 확률로 예측함

VGG-16 개요

VGG-16은

- 2014년 ILSVRC 대회에서 준우승한 CNN 모델
- 옥스퍼드 대학 **VGG(Visual Geometry Group)** 연구팀 개발

특징

- 구조는 단순
 - 깊이는 매우 깊음
 - 이후 CNN 설계의 **기준 모델**
-

GoogleNet vs VGG-16 (배경 이해)

- GoogleNet: Inception module 사용
 - 다양한 필터 크기를 병렬로 적용
 - 구조 복잡
- VGG-16:
 - **3×3 필터만 반복 사용**
 - 구조 단순 → 연구/확장에 유리

성능은 GoogleNet이 약간 더 좋았지만

VGG-16이 더 많이 사용됨

VGG-16 구조 핵심

계층 구성

- 총 16개 계층

- 합성곱 계층 13개
- 완전 연결 계층 3개

주요 특징

- 모든 합성곱 필터 크기 = 3×3
- ReLU 활성화
- Max Pooling 사용

AlexNet vs VGG-16 구조 비교

항목	AlexNet	VGG-16
합성곱 필터	11×11 , 5×5 혼합	3×3 만 사용
네트워크 깊이	상대적으로 얇음	매우 깊음
최대 채널 수	384	512
파라미터 수	약 6,100만	약 1억 3천만
성능	acc@1 $\approx 56\%$	acc@1 $\approx 71\%$

깊고 작은 필터 = 더 강한 표현력

수용 영역(Receptive Field, RF)

수용 영역이란 한 뉴런이 입력 이미지에서 영향을 받는 영역의 크기

(1) 큰 필터 사용 (AlexNet)

- 11×11 필터
- 한 번에 넓은 영역을 봄
- 전역 특징(Global feature)에 유리
- 하지만 세밀한 지역 특징은 놓칠 수 있음

(2) 작은 필터 반복 사용 (VGG-16)

- 3×3 필터를 여러 번 적용
- 결과적으로 큰 수용 영역 확보

- 지역 특징(Local feature) + 전역 특징 모두 학습 가능

7×7 필터 vs 3×3 필터 3회 비교

방식	파라미터 수	활성화 함수
7×7 필터 1회	49개	1번
3×3 필터 3회	27개	3번

3×3 필터 반복의 장점:

- 파라미터 감소
- 비선형성 증가 (ReLU 여러 번)
- 표현력 향상

VGG-16 성능이 더 좋은 이유

1. 더 깊은 구조
2. 작은 필터 반복 → 비선형성 증가
3. 채널 수 증가 (최대 512)
4. 더 많은 정보 저장 가능

결과:

- AlexNet 오류율 ≈ 16%
- VGG-16 오류율 ≈ 7%

Pet 데이터셋 구조

```
pet/
├─ train/
│   ├─ dog/
│   └─ cat/
└─ test/
```

```
|— dog/  
|— cat/
```

- 학습 데이터 $\approx 8,000$ 장
- 테스트 데이터 $\approx 2,000$ 장
- **ImageFolder** 사용 가능 구조

이미지 전처리(Transform) 이유

VGG-16은 **224×224** 크기 이미지로 학습됨

전처리 흐름

```
Resize(256)  
→ CenterCrop(224)  
→ ToTensor()  
→ Normalize(mean, std)
```

왜 Resize 후 Crop?

- 객체가 중앙에 있을 확률이 높음
- 배경 노이즈 제거
- 객체 크기 유지

정규화(Normalize)

```
Normalize(  
    mean=[0.48235, 0.45882, 0.40784],  
    std=[1/255, 1/255, 1/255]  
)
```

- 픽셀 값을 모델이 학습한 분포와 맞춤
- 학습 안정성 향상

ImageFolder + DataLoader

ImageFolder

- 폴더 이름 → 자동 레이블
- 이미지 + 레이블 자동 매핑

DataLoader

- 배치 단위로 데이터 제공
- shuffle, batch_size, drop_last 설정

데이터 시각화 목적

- 전처리가 올바르게 적용됐는지 확인
- 레이블 매칭 오류 확인
- 데이터 품질 점검

시각화 시 처리

- Tensor → NumPy
- (C, H, W) → (H, W, C)
- 정규화 역연산 후 uint8 변환

VGG-16 모델 로드

```
model = torchvision.models.vgg16(  
    weights="VGG16_Weights.IMAGENET1K_V1"  
)
```

- ImageNet으로 사전 학습된 모델
- 1000개 클래스 분류 가능

VGG-16 모델 정보

항목	값
acc@1	71.6%
acc@5	90.3%
파라미터 수	약 1억 3천만
GFLOPS	15.47
파일 크기	약 528MB

AlexNet보다 크고 무겁지만 성능 우수

VGG-16 모델 구조 확인

```
model = models.vgg16(weights="VGG16_Weights.IMAGENET1K_V1")
print(model)
```

VGG-16의 전체 구성

VGG-16은 크게 **3부분**으로 나뉜다.

(1) features (특징 추출기)

- 여러 개의 **Conv2d + ReLU + MaxPool**
- 입력 이미지에서 **저수준 → 고수준** 특징 추출
- 사전 학습된 핵심 부분

(2) avgpool

```
AdaptiveAvgPool2d(output_size=(7, 7))
```

- 입력 크기와 상관없이 **7×7 고정**
- 완전연결 계층에 넣기 위한 정규화된 크기 생성

(3) classifier (분류기)

```
Linear(25088 → 4096)
Linear(4096 → 4096)
Linear(4096 → 1000)
```

- ImageNet 기준 **1000개** 클래스 출력
- 마지막 `out_features=1000` 이 분류 클래스 수

이 상태는 **ImageNet 전용 모델**

VGG-16 미세 조정(Fine-tuning) 개념

문제 상황

- 현재 목표: **dog vs cat (2클래스)**
- 기존 VGG-16 출력: **1000 클래스**

마지막 분류 계층만 수정하면 됨

분류기 수정 코드

```
model.classifier[6] = nn.Linear(
    4096,
    len(train_dataset.classes) # = 2
)
```

핵심 포인트

- `classifier[6]` → 마지막 선형 계층
- 앞의 계층들은 **사전 학습 가중치 유지**
- 마지막 계층만 **새로 학습**

이것이 **전이 학습(Transfer Learning)** 의 핵심

왜 마지막 계층만 바꾸는가?

- 앞부분(features):
 - 엣지, 질감, 형태 같은 **일반적 시각 특징**
 - 거의 모든 이미지 문제에 재사용 가능
- 뒷부분(classifier):

- 문제 특화(클래스 의존)
- dog/cat에 맞게 다시 학습 필요

적은 데이터로도 안정적인 학습 가능

VGG-16 모델 학습 전체 흐름 (예제 8.9)

(1) 데이터 로딩

```
train_dataset = ImageFolder("../datasets/pet/train", transform=...)
test_dataset = ImageFolder("../datasets/pet/test", transform=...)
```

- 폴더 이름 = 클래스 라벨
 - `ImageFolder` 는 자동 라벨링
-

(2) DataLoader

```
Dataloader(dataset, batch_size=4, shuffle=True)
```

- 미니배치 학습
 - 데이터 섞기(shuffle)로 일반화 성능 향상
-

(3) 손실 함수 & 최적화

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.0001)
```

- CrossEntropyLoss:
 - 다중 클래스 분류 표준 손실
 - SGD:
 - 단순하지만 안정적
-

(4) 학습 루프

```
output = model(images)
loss = criterion(output, classes)

loss.backward()
optimizer.step()
```

- 순전파 → 손실 계산 → 역전파 → 가중치 업데이트
- epoch마다 cost 감소 확인

출력 예시:

```
Epoch 1 Cost: 0.276
Epoch 5 Cost: 0.028
```

모델 평가 (acc@1)

```
outputs_classes = torch.argmax(probs, dim=-1)
accuracy += int(torch.eq(classes, outputs_classes).sum())
```

acc@1 의미

- Top-1 예측이 정답인 비율
- dog/cat 문제 → 가장 직관적인 정확도

$$\text{acc@1} = (\text{정답 개수} / \text{전체 샘플 수}) \times 100$$

결과:

```
acc@1 : 97.77%
```

소규모 데이터에서도 사전학습 + 미세조정 효과 확인

VGG-16 학습 결과 해석

- 에폭 수가 적어도 cost 안정적으로 감소

- 정확도 매우 높음
- 이유:
 - VGG-16은 이미 개·고양이 이미지로 사전 학습된 모델

따라서 적은 데이터 + 짧은 학습으로도 좋은 성능

ResNet 등장 배경

VGG의 한계

- 깊어질수록:
 - 기울기 소실(Vanishing Gradient)
 - 학습이 오히려 안 되는 현상 발생

단순히 계층을 깊게 쌓는 것만으로는 성능이 증가하지 않음

기울기 저하 문제 (Degradation Problem)

- 더 깊은 모델이:
 - 얇은 모델보다 훈련 오차가 더 큼
 - 이는:
 - 과적합 문제가 아니라
 - 최적화 자체가 어려워지는 문제
-

ResNet의 핵심 아이디어: 잔차 학습 (Residual Learning)

기존 학습

$H(x)$

- 입력 $x \rightarrow$ 출력 $H(x)$ 를 직접 학습

ResNet 학습

$$F(x)=H(x)-xF(x) = H(x) - x$$

즉, 출력과 입력의 차이(Residual)만 학습

잔차 연결 (Residual / Skip Connection)

기본 수식

$$y=F(x,W)+xy = F(x, W) + x$$

- x : 이전 계층 출력
- $F(x)$: 현재 계층에서 학습한 변화량
- 덧셈으로 정보 보존

효과:

- 기울기가 직접 전달됨
 - 깊어져도 학습 가능
-

차원이 다를 때

$$y=F(x,W)+Wsxy = F(x, W) + W_s x$$

- W_s : 1×1 convolution
 - 차원 맞추용 선형 변환
-

ResNet 기본 블록 vs 병목 블록

(1) 기본 블록 (Basic Block)

- $3 \times 3 \rightarrow 3 \times 3$
- ResNet-18, ResNet-34 사용

(2) 병목 블록 (Bottleneck Block)

- $1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$
- 채널 ↓ → 연산 → 채널 ↑
- ResNet-50 이상 사용

장점:

- 연산량 감소
- 깊은 네트워크 구성 가능

ResNet의 의의

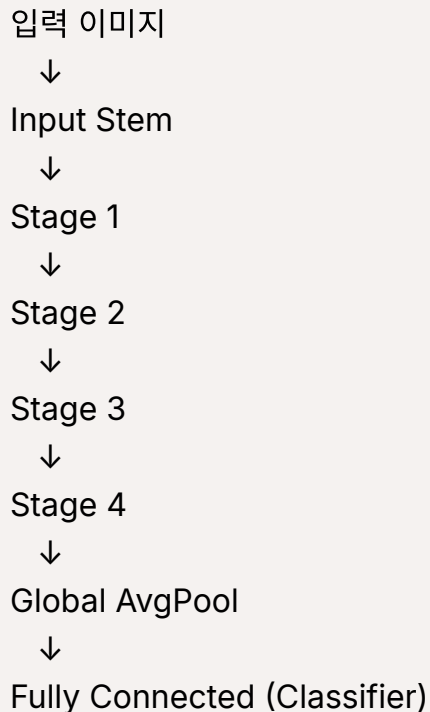
- 152-layer까지 안정적 학습 가능
- ILSVRC 2015 우승
- 이후 CNN 구조의 **표준**

오늘날 대부분의 비전 모델은

ResNet 아이디어를 기반으로 발전

ResNet 전체 구조

ResNet은 하나의 입력 줄기(input stem) 와 4개의 스테이지(stage) 로 구성된다.



```
graph TD; A[입력 이미지] --> B[Input Stem]; B --> C[Stage 1]; C --> D[Stage 2]; D --> E[Stage 3]; E --> F[Stage 4]; F --> G[Global AvgPool]; G --> H[Fully Connected (Classifier)];
```

입력 이미지
↓
Input Stem
↓
Stage 1
↓
Stage 2
↓
Stage 3
↓
Stage 4
↓
Global AvgPool
↓
Fully Connected (Classifier)

Input Stem (입력 줄기)

모든 ResNet 모델이 동일하게 사용하는 구조

7×7 Conv (stride=2, padding=3)
→ BatchNorm
→ ReLU
→ 3×3 MaxPool (stride=2)

역할

- 입력 이미지 크기: 224×224
- 빠르게 해상도 ↓, 채널 수 ↑
- 초기 저수준 특징(edge, texture) 추출

Stage 개념 (ResNet의 핵심 구조)

ResNet은 **Stage** 단위로 구조를 반복한다.

Stage	출력 크기	채널 수
Stage 1	56×56	64
Stage 2	28×28	128
Stage 3	14×14	256
Stage 4	7×7	512

Stage가 바뀔 때

- spatial size ↓ (stride=2)
- channel size ↑ (64 → 128 → 256 → 512)

ResNet 모델별 차이 (표의 핵심)

블록 개수 차이

모델	블록 타입	Stage별 블록 수
ResNet-18	BasicBlock	[2, 2, 2, 2]
ResNet-34	BasicBlock	[3, 4, 6, 3]

모델	블록 타입	Stage별 블록 수
ResNet-50	Bottleneck	[3, 4, 6, 3]
ResNet-101	Bottleneck	[3, 4, 23, 3]
ResNet-152	Bottleneck	[3, 8, 36, 3]

깊어질수록 Bottleneck 사용

BasicBlock (ResNet-18 / 34)

구조 요약

```

3×3 Conv
→ BN
→ ReLU
→ 3×3 Conv
→ BN
→ + shortcut
→ ReLU

```

코드 핵심

```

out += self.shortcut(x)
out = self.relu(out)

```

특징

- expansion = 1
- 입력 채널 수 = 출력 채널 수
- 구조 단순, 연산량 비교적 큼
- 얇은 ResNet에 적합

Shortcut의 역할

```

if stride != 1 or inplanes != planes:

```

shortcut = Conv1×1 + BN

Shortcut이 필요한 경우

- stride=2로 해상도 줄일 때
- 채널 수가 바뀔 때

입력과 출력 차원을 강제로 맞춤

5. Bottleneck Block (ResNet-50 이상)

구조 요약

1×1 Conv (채널 ↓)
→ 3×3 Conv
→ 1×1 Conv (채널 ↑)
→ + shortcut
→ ReLU

expansion = 4

- planes = 64 → 출력 채널 = 256
- 마지막 1×1 Conv에서 채널 수 폭증

왜 Bottleneck을 쓰는가?

- 깊은 네트워크에서
 - 연산량 폭증
 - 메모리 부담 증가
- 1×1 Conv로
 - 연산량 감소
 - 표현력 유지

깊은 ResNet의 핵심 기술

ResNet 클래스 구현 구조

핵심 멤버 변수

```
self.inplanes = 64
```

- 현재 feature map의 채널 수 추적
- stage가 바뀔 때 갱신됨

Stage 생성 메서드: `_make_layer`

```
def _make_layer(self, block, planes, num_blocks, stride):
```

동작 방식

1. 첫 블록

- stride 적용 (보통 2)
- 해상도 감소

2. 나머지 블록

- stride=1
- 동일 크기 유지

```
layers.append(block(self.inplanes, planes, stride))  
self.inplanes = planes * block.expansion
```

첫 블록만 **stride ≠ 1**

ResNet forward 흐름 (순전파)

```
out = self.stem(x)  
out = self.stage1(out)  
out = self.stage2(out)  
out = self.stage3(out)
```

```
out = self.stage4(out)
out = self.avgpool(out)
out = torch.flatten(out, 1)
out = self.fc(out)
```

핵심 포인트

- `AdaptiveAvgPool2d(1,1)`
 - 입력 크기 상관없이 처리 가능
- `flatten` 후 FC 계층으로 분류

직접 구현 ResNet vs torchvision ResNet

비교 코드

```
resnet34 = ResNet(BasicBlock, [3,4,6,3], 1000)
torch_model = models.resnet34(weights=...)
```

결과

```
21797672
21797672
```

파라미터 수 완전히 동일

의미:

- 우리가 구현한 ResNet 구조가
- 논문 및 공식 구현과 **정확히 동일**

딥러닝의 문제점: Black Box

- CNN, ResNet 같은 딥러닝 모델은 **정확도는 높지만**
- *"왜 이 이미지를 이렇게 분류했는지"* 설명하기 어렵다
- 특히 **의료, 자율주행, 보안** 분야에서는 **설명 가능성(XAI)** 이 매우 중요

그래서 **모델이 이미지의 어느 부분을 보고 판단했는지 시각화**하는 기법이 등장

Class Activation Map (CAM)

CAM이란?

특정 클래스 예측에 가장 크게 기여한 이미지 영역을 히트맵으로 시각화하는 방법

- CNN 기반 이미지 분류 모델에서 사용
- “이 이미지에서 어디를 보고 airplane이라고 판단했나?”를 보여줌

CAM이 가능한 모델 구조

CAM은 아래 구조를 반드시 만족해야 함

Conv Layers → Global Average Pooling (GAP) → Fully Connected (FC)

GAP 없는 일반 CNN → CAM 불가

구조 바꾸고 재학습 필요

CAM 생성 원리

1. 마지막 합성곱 계층에서 특징 맵 추출

→ 크기: (K, H, W) (예: $512 \times 7 \times 7$)

2. **GAP**으로 채널별 평균 → $(K,)$

3. FC 계층에서 클래스별 가중치 사용

- 클래스 c 에 대한 가중치: w_k^c

4. 각 채널 특징 맵에 가중치 곱해서 합산

- $f_k(i,j)$: k 번째 채널의 (i,j) 위치 값
- w_k^c : 클래스 c 에 대한 FC 가중치

값이 클수록 그 위치가 해당 클래스 분류에 중요

CAM의 장단점

장점

- 직관적이고 계산 간단
- 시각화 결과 이해 쉬움

단점 (치명적)

- **GAP** 구조 필수
- 기존 모델 구조 변경 + 재학습 필요
- 범용성 낮음

이 한계를 해결한 게 **Grad-CAM**

Grad-CAM (Gradient-weighted CAM)

Grad-CAM이란?

기울기(gradient)를 이용해 어떤 CNN 구조에서도 사용 가능한 CAM

- CAM의 확장 버전
 - **GAP 필요 없음**
 - ResNet, VGG, DenseNet 등 거의 모든 CNN에 적용 가능
-

핵심 아이디어

“이 클래스 점수가 특정 특징 맵에 얼마나 민감한가?”

즉,

- 출력 클래스 점수 y^c
 - 마지막 합성곱 특징 맵 f_k
 - 이 둘의 **기울기(gradient)** 를 이용
-

Grad-CAM 전체 흐름 요약

입력 이미지

→ CNN forward

→ 예측 클래스 선택

- backward (gradient 계산)
- 채널 중요도 α 계산
- 가중합 + ReLU
- heatmap 생성
- 원본 이미지에 overlay

ResNet + Grad-CAM 연결 이해

ResNet 구조:

Conv1 → Layer1~4 → AvgPool → FC

Grad-CAM에서는:

- AvgPool, FC 이전
- 즉 Layer4 출력 ($512 \times 7 \times 7$) 사용

```
features = nn.Sequential(*list(model.children())[:-2])
```

- 마지막 conv feature map 확보
- FC 가중치
- gradient 기반 중요도 계산

결과 해석 방법

빨간색 영역

- 해당 클래스 예측에 강하게 기여
- 모델이 “집중해서 본 부분”

파란색 영역

- 거의 영향 없음

예:

- airplane → 동체, 날개

- dog → 얼굴, 몸통
 - bus → 차량 전체 윤곽
-

왜 Grad-CAM이 필요한가? (XAI 배경)

딥러닝의 문제

- CNN은 정확도는 높지만 내부 동작이 불투명(Black Box)
- “왜 이 이미지를 비행기라고 판단했는가?”를 설명하기 어려움

XAI (Explainable AI)

- 모델의 판단 근거를 사람이 이해 가능하게 설명
 - 이미지 분류에서는 어떤 이미지 영역이 분류에 영향을 줬는지 시각화하는 방식이 중요
-

CAM (Class Activation Map)

핵심 아이디어

“이 클래스 점수를 만들 때, 어떤 공간 위치의 특징이 많이 기여했는가?”

전제 조건

CAM은 특정 구조의 CNN에서만 가능

- 마지막 합성곱 층
- **Global Average Pooling (GAP)**
- 그 뒤에 **Fully Connected (FC)**

즉:

Conv feature map → GAP → FC → class score

CAM 생성 원리 (수식 8.3)

[

$$L^c(i,j) = \sum_k w_k^c f_k(i,j)$$

]

- $(f_k(i,j))$: 마지막 합성곱 층의 k 번째 채널 feature map
- (w_k^c) : 클래스 c 에 대한 FC 가중치
- 결과: $[H \times W]$ 크기의 heatmap

FC 가중치를 이용해 feature map을 가중합

CAM의 한계

- GAP 구조가 필수
- 구조가 다르면 모델을 수정하거나 재학습해야 함
- 공간 정보가 일부 손실됨

이 한계를 해결하기 위해 **Grad-CAM** 등장

Grad-CAM (Gradient-weighted CAM)

핵심 차이점 (CAM vs Grad-CAM)

구분	CAM	Grad-CAM
가중치	FC 가중치 (w_k^c)	Gradient 기반 가중치 (α_k^c)
GAP 필요	필요	불필요
적용 범위	제한적	모든 CNN 가능
방식	구조 의존	기울기(gradient) 기반

Grad-CAM 가중치 계산

- (y^c) : 클래스 c 의 출력 점수
- $(f_k(i,j))$: k 번째 feature map
- (α_k^c) : 해당 채널이 클래스 c 에 얼마나 중요한지

Gradient의 전역 평균 = 채널 중요도

Grad-CAM 최종 맵

- ReLU를 쓰는 이유:
 - 양의 기여만 시각화
 - 음수는 해당 클래스와 무관한 정보

Grad-CAM 계산 흐름

1. 입력 이미지 → 모델 **forward**
 2. 예측된 클래스 (c) 선택
 3. 해당 클래스 점수에 대해 **backward**
 4. 마지막 합성곱 층에서:
 - feature map 저장
 - gradient 저장
 5. gradient를 공간 평균 → (α_k^c)
 6. feature map 가중합 + ReLU
 7. 원본 이미지 크기로 interpolate
 8. heatmap으로 시각화
-

Hook을 쓰는 이유

문제

- PyTorch는 중간 **layer** 출력과 **gradient**를 자동 저장하지 **않음**

해결

Hook

- 특정 layer에서:
 - 순전파 결과(**feature map**) 저장
 - 역전파 **gradient** 저장
-

사용한 Hook 종류

Hook	역할
<code>register_forward_hook</code>	feature map 저장
<code>register_full_backward_hook</code>	gradient 저장

ResNet-18 기준:

- 마지막 합성곱 층: `layer4[-1].conv2`

Grad-CAM 클래스 구조

`__init__`

- 모델을 eval 모드로 설정
- 지정한 layer에 hook 등록

`forward_hook`

- 마지막 합성곱 층 출력 $\rightarrow$ `self.feature_map`

`backward_hook`

- `gradient` $\rightarrow$ `self.gradient`
-

`__call__(x)` 내부 로직

1. 모델 forward
 2. 예측 클래스 index 추출
 3. one-hot 벡터 생성
 4. backward 실행
 5. gradient 평균 $\rightarrow (\alpha_k^c)$
 6. feature map 가중합
 7. ReLU 적용
 8. Grad-CAM 반환
-